JOHN VON NEUMANN UNIVERSITY | GAMF
Faculty of Engineering and Computer Science | Informatics Department

Forex Horizon

Java Applications - Lecture Homework

Browser-based desktop application
Java 25 | Spring Boot 3.5 | Thymeleaf | Bootstrap 5

Authors (replace with both students' full names and Neptun codes):
Student One (NEPTUN1)
Student Two (NEPTUN2)

GitHub URL (public repository — mandatory):
https://github.com/<org-or-user>/forex-horizon

Executable: homework.jar in the repository root
Command: java -jar homework.jar (Java 25 required)

This document maps every mandatory and extra task from Java applications-Lecture-Homework-THE TASK.docx to the Forex Horizon implementation. Insert real screenshots, GitHub URL, author names, and OANDA practice-account access before submitting to Teams as Name-NeptunCode.pdf. Each group member must submit the PDF. Do not upload other files to Teams.

Contents

- 1. Assignment summary and scoring
- 2. Team, GitHub URL, and division of work
- 3. Environment, how to run, and homework.jar
- 4. Application architecture
- 5. Free responsive theme (3 points)
- 6. Home menu — company front page
- 7. SOAP menu — MNB web service and Chart.js (4 points)
- 8. Forex client — OANDA v20 practice API
- 8.1 Forex-account (1 p)
- 8.2 Forex-ActPrice (3 p)
- 8.3 Forex-HistPrice (3 p)
- 8.4 Forex-Open (3 p)
- 8.5 Forex-Pos (1 p)
- 8.6 Forex-Close (2 p)
- 9. Extra: Test menu, JUnit and MockMvc (+5 points)
- 10. GitHub version control and project method (2 + 3 points)
- 11. Security: secrets, practice account, .gitignore
- 12. Appendix — OANDA evaluator access and screenshot checklist

1. Assignment summary and scoring

Course: Java applications — Lecture homework (Java Spring Boot). Instructor materials: Java applications-Lecture-Homework-THE TASK.docx and GitHub-description-2026-07.docx (Dr. Zoltán Subecz, John von Neumann University, Kecskemét).

The assignment requires a pair of students to build a local browser application with Spring Boot. Working in a group of two is mandatory unless the teacher authorises an individual exception. If one student works alone with permission, two GitHub accounts must still be used so the collaborative GitHub component can be graded. The educational goal is corporate-style collaboration on a larger task.

Mandatory platform constraints that give 0 points if missed: Java 25 (not 21); an executable homework.jar in the GitHub root that starts with java -jar homework.jar; a public GitHub repository; documentation of at least 10 pages in PDF, submitted only to Teams, by every group member, with filename Name-NeptunCode.pdf.

Scoring table from THE TASK

Task	Pts	Where implemented in Forex Horizon
Free responsive theme; name the theme in this PDF	3	Bootstrap 5.3.3 CDN + static/css/site.css
Home: company site; names + Neptun on every page	incl.	index.html, fragments.html footer
SOAP client: MNB rates, dates, table, Chart.js	4	MnbService, PageController /soap, soap.html
Forex-account: print account information	1	OandaService.account(), GET /forex/account
Forex-ActPrice: instrument dropdown, actual price	3	OandaService.price(), GET /forex/price
Forex-HistPrice: instrument + granularity, last 10	3	OandaService.candles(), GET /forex/history
Forex-Open: instrument + quantity Long/Short	3	OandaService.open(), POST /forex/open
Forex-Pos: open positions in a table	1	OandaService.positions(), GET /forex/positions
Forex-Close: TradeId number, close at market	2	OandaService.close(), POST /forex/close
GitHub, public, ≥5 incremental commits, own names	2	Public repo; see section 10
Project method: who created which part	3	Section 2 of this document

PDF documentation ≥10 pages, screenshots, GitHub URL, OANDA login	mand.	This file
Extra: Test menu + JUnit + MockMvc	+5	CalculatorService, /test, *Test.java
Extra: React CRUD + H2 (not implemented)	+5	Out of scope for this delivery

Total of the mandatory application items is 25 points. The Test calculator is implemented to collect the extra 5 points. The optional React + H2 CRUD menu is not part of Forex Horizon, matching the project README (no React menu).

2. Team, GitHub URL, and division of work

Fill the table before submission. THE TASK requires that the documentation state which team member created which part. GitHub must show the same split through commits from two accounts. Use your own name on Git so the uploader can be identified.

Field	Value to fill
Student A name + Neptun	_____
Student B name + Neptun	_____
Public GitHub URL	https://github.com/_____/forex-horizon
Footer authors env APP_AUTHORS	First Last (ABC123) and Second Last (DEF456)

Suggested contribution split (edit to match real commits)

Part of the assignment	Owner	Evidence in the repo
Maven skeleton, Application.java, properties	Student A	pom.xml, Application.java
Responsive theme, Home, fragments, CSS	Student A	templates/, site.css
SOAP MNB client, soap.html, Chart.js	Student A	MnbService.java, soap.html
OANDA client and six Forex pages	Student B	OandaService.java, forex.html
Test calculator, JUnit, MockMvc	Student B	CalculatorService, *Test.java
JAR packaging, README, .gitignore	Both	pom finalName, homework.jar
This documentation PDF	Both	docs/

app.authors is injected on every page via PageController.@ModelAttribute and printed in the Thymeleaf footer fragment. That satisfies: “Enter the names and Neptun codes of the creators at the bottom of every page.”

```
application.properties:
  app.authors=${APP_AUTHORS:Student One (NEPTUN1) and Student Two (NEPTUN2)}
PageController.common(Model): model.addAttribute("authors", authors);
fragments.html footer: Created by … th:text="${authors}"
```

3. Environment, how to run, and homework.jar

Mandatory Java 25

THE TASK states Java 25 is mandatory. pom.xml sets <java.version>25</java.version> and maven-compiler-plugin <release>25</release>. Spring Boot parent is 3.5.7. Maven 3.9+ is required. A Java 21 JDK will not compile this application.

Dependencies (pom.xml)

- spring-boot-starter-web — embedded Tomcat, REST client, MVC
- spring-boot-starter-thymeleaf — server-side HTML
- spring-boot-starter-validation — form/parameter support
- spring-boot-starter-test — JUnit 5, Mockito, MockMvc (test scope)
- spring-boot-maven-plugin with <finalName>homework</finalName> so the fat JAR is homework.jar

Run locally

Secrets stay in environment variables, never in source. Copy .env.example values into the shell. OANDA_BASE_URL defaults to the practice host https://api-fxpractice.oanda.com.

```
export OANDA_ACCOUNT_ID="your_practice_account_id"
export OANDA_API_TOKEN="your_practice_api_token"
export APP_AUTHORS="First Last (ABC123) and Second Last (DEF456)"
mvn clean test
mvn clean package
cp target/homework.jar ./homework.jar
java -jar homework.jar
# open http://localhost:8080
```

server.port uses ${PORT:8080}. Thymeleaf cache is disabled for development. After package, THE TASK requires uploading homework.jar to the GitHub project root so the evaluator can download it and run java -jar homework.jar on Java 25. Failure of that command is a mandatory-item failure (0 points for the solution).

[SCREENSHOT PLACEHOLDER] Figure 1 — Terminal: java -version showing 25, then java -jar homework.jar starting Tomcat on 8080.

Replace this box with a real screenshot of the running application (or of mvn test).

[SCREENSHOT PLACEHOLDER] Figure 2 — Browser address bar at http://localhost:8080 after a successful start.

Replace this box with a real screenshot of the running application (or of mvn test).

4. Application architecture

Forex Horizon is a classic Spring MVC layered application: one @SpringBootApplication, one

@Controller that maps every menu URL, three @Service classes for calculator / MNB SOAP / OANDA REST, Thymeleaf templates for HTML, and static CSS. There is no database and no React front end.

Request flow

- Browser GET/POST → PageController mapping.
- Controller puts titles, form values, and results on the Spring Model.
- CalculatorService / MnbService / OandaService perform logic or HTTP calls.
- Thymeleaf renders index.html, soap.html, forex.html, or test.html, always wrapping fragments :: nav and footer.
- Errors become flash-style model attributes (error) and are shown as Bootstrap alerts; the site does not crash.

Source map

File	Role
Application.java	@SpringBootApplication; main()
PageController.java	All menus: /, /soap, /forex/*, /test
CalculatorService.java	Four-operation BigDecimal calculator
MnbService.java	SOAP GetExchangeRates client + XML parse
OandaService.java	OANDA v20 RestClient (Bearer token)
application.properties	Port, authors, MNB URL, OANDA settings
templates/*.html	Home, SOAP, Forex, Test, nav/footer
static/css/site.css	Custom theme on top of Bootstrap 5
CalculatorServiceTest.java	JUnit unit tests
PageControllerTest.java	MockMvc slice tests

Shared model attributes

PageController.common() runs for every request: authors, whether OANDA is configured (token and account non-blank), the instrument list EUR_USD, GBP_USD, USD_JPY, EUR_GBP, AUD_USD, USD_CHF, and granularities M5, M15, H1, H4, D, W. Forex pages reuse one template (forex.html) with a page flag (account, price, history, open, positions, close) so the assignment's six Forex menus stay consistent.

5. Free responsive theme (3 points)

THE TASK: “Find and apply a free responsive theme to your pages. Describe in the documentation which theme you chose.” Help links in the assignment point to HTML5UP, ThemeWagon, and similar galleries.

Chosen theme / stack: Bootstrap 5.3.3 (free MIT-licensed responsive CSS/JS framework from https://getbootstrap.com / jsDelivr CDN) plus a custom “Forex Horizon” skin in src/main/resources/static/css/site.css. Bootstrap is used instead of a third-party HTML5UP multi-page template because Spring Thymeleaf needs fragment-based layouts (nav, footer) rather than a static one-page theme. Bootstrap 5 is a free responsive HTML/CSS/JS theme system: navbar-expand-lg, grid (container, row, col-md-*), form-select, form-control, tables, alerts, and a hamburger toggler for narrow viewports.

What was applied

- CDN: bootstrap@5.3.3/dist/css/bootstrap.min.css and bootstrap.bundle.min.js in fragments.html.
- Viewport meta on every page: width=device-width, initial-scale=1.
- Navbar collapses on mobile (navbar-toggler, data-bs-target=#siteNav).
- Home uses a hero block and three col-md-4 cards; forms use Bootstrap grid.
- Custom variables in site.css: navy #071b32, cyan #13d1cb; hero gradient; card class .info; JSON panel .api for OANDA responses.

This is documented here because the 3 points are awarded only if the PDF names the theme. If the team later swaps in an HTML5UP stylesheet, update this section with the exact theme name (for example “HTML5 UP — Forty”) and keep Bootstrap grid classes or restyle the fragments accordingly.

[SCREENSHOT PLACEHOLDER] Figure 3 — Home page on a wide desktop window (hero + three cards + footer authors).

Replace this box with a real screenshot of the running application (or of mvn test).

[SCREENSHOT PLACEHOLDER] Figure 4 — Same Home page at a phone width, showing the collapsed navbar toggler.

Replace this box with a real screenshot of the running application (or of mvn test).

6. Home menu — company front page

URL GET / maps to PageController.home() which returns the view name “index”. index.html introduces the fictional company Forex Horizon (currency intelligence): Hungarian central-bank reference rates and an OANDA practice trading desk. Call-to-action links go to /soap, /forex/account, and /test.

Main-page menu (mandatory items)

fragments.html defines nav: Home, SOAP, Forex dropdown (six items named as in THE TASK: Forex-account, Forex-ActPrice, Forex-HistPrice, Forex-Open, Forex-Pos, Forex-Close), and Test. There is no React menu. Every HTML page includes th:replace="~{fragments :: nav}" and the footer.

[SCREENSHOT PLACEHOLDER] Figure 5 — Top navigation with Forex dropdown expanded, matching the assignment menu names.

Replace this box with a real screenshot of the running application (or of mvn test).

7. SOAP menu — MNB web service and Chart.js (4 points)

THE TASK: create a SOAP client for the Hungarian National Bank web service; select currency in a drop-down; start and end date with date inputs; print the data; draw a chart with https://www.chartjs.org/.

User interface (soap.html)

- GET /soap shows the empty form. POST /soap submits currency, start, end.
- Currency drop-down: EUR, USD, GBP, CHF, JPY, CZK, PLN, RON.
- Start/end use HTML type=date as required.
- Results: HTML table (date, currency, HUF rate) and a Chart.js line chart (#ratesChart).
- Empty result set shows a warning; SOAP/network errors show an alert.

Where it is implemented

MnbService is a Spring @Service. It builds RestClient against app.mnb.url (https://www.mnb.hu/arfolyamok.asmx). history(currency, from, to) validates that start ≤ end and that the currency is in the allowed set. It posts a SOAP 1.1 envelope for GetExchangeRates with SOAPAction http://www.mnb.hu/webservices/GetExchangeRates and Content-Type text/xml;charset=utf-8.

```
Inner SOAP body (namespaces shortened):
<GetExchangeRates xmlns="http://www.mnb.hu/webservices/">
  <startDate>YYYY-MM-DD</startDate>
  <endDate>YYYY-MM-DD</endDate>
  <currencyNames>EUR</currencyNames>
</GetExchangeRates>
```

The SOAP result wraps a nested XML string (GetExchangeRatesResult). The service parses it with a namespace-aware DocumentBuilderFactory hardened against XXE (doctype disallowed, external entities off). Each Day/Rate node with matching curr becomes a Rate record(date, currency, value). Comma decimals are normalised to dots; if unit ≠ 1 the rate is divided by unit so the chart uses one HUF per unit of currency.

PageController.rates() puts rates, labels (dates), and values (doubles) on the model. soap.html inlines those arrays into Chart.js: type line, tension 0.2, label “HUF reference rate”. Chart.js 4.4.7 is loaded from CDN.

[SCREENSHOT PLACEHOLDER] Figure 6 — SOAP form filled (e.g. EUR, a one-month range) before submit.

Replace this box with a real screenshot of the running application (or of mvn test).

[SCREENSHOT PLACEHOLDER] Figure 7 — Table of MNB rates plus the Chart.js line chart after a successful SOAP call.

Replace this box with a real screenshot of the running application (or of mvn test).

8. Forex client — OANDA v20 practice API

THE TASK requires an OANDA Forex client with six menus. Implementation: OandaService using Spring RestClient, Jackson JsonNode, and Authorization: Bearer <token>. Base URL, token, and account come from application.properties (environment variables OANDA_BASE_URL, OANDA_API_TOKEN, OANDA_ACCOUNT_ID). configured() is false when token or account is blank; Forex pages then show a Bootstrap warning instead of calling the API. Live trading is avoided by keeping the default host on api-fxpractice.oanda.com. THE TASK's help chapter "Creating Forex trader client" is realised here with HTTP rather than a SOAP stub.

Shared Forex view

PageController.run() sets title, page key, and either data or error, then returns view "forex". forex.html shows instrument/granularity/units forms when needed, a positions table when the payload is an array of trades, and a pretty-printed JSON block (data.toPrettyString()) so account and price responses remain inspectable.

8.1 Forex-account (1 point)

GET /forex/account → OandaService.account() → GET /v3/accounts/{accountId}/summary, then JSON path "account". The page prints account information (id, currency, NAV, margin, open trade count, and related fields returned by OANDA).

[SCREENSHOT PLACEHOLDER] Figure 8 — Forex-account page with practice account summary JSON.

Replace this box with a real screenshot of the running application (or of mvn test).

8.2 Forex-ActPrice (3 points)

GET /forex/price?instrument=EUR_USD (default EUR_USD). Drop-down of instruments as required. OandaService.price() calls GET /v3/accounts/{id}/pricing?instruments=… and returns path "prices" (bids/asks, tradeable flag, time).

[SCREENSHOT PLACEHOLDER] Figure 9 — Forex-ActPrice: instrument selected and actual price printed.

Replace this box with a real screenshot of the running application (or of mvn test).

8.3 Forex-HistPrice (3 points)

GET /forex/history with instrument and granularity drop-downs (granularity default D = daily). OandaService.candles() calls GET /v3/instruments/{instrument}/candles?count=10&price=M&granularity=… which is the last 10 mid-price candles, matching "print the last 10 historical prices." The view states that up to 10 candles are shown.

[SCREENSHOT PLACEHOLDER] Figure 10 — Forex-HistPrice: EUR_USD, granularity D, ten candles.

Replace this box with a real screenshot of the running application (or of mvn test).

8.4 Forex-Open (3 points)

GET /forex/open shows the form. POST /forex/open sends instrument and units (number input). THE TASK: positive quantity = Long, negative = Short; open at market price. OandaService.open() rejects units == 0 and POSTs /v3/accounts/{id}/orders with a MARKET order, timeInForce FOK, positionFill DEFAULT, units as string (sign encodes buy/sell).

[SCREENSHOT PLACEHOLDER] Figure 11 — Forex-Open form (instrument + quantity) and the order create response.

Replace this box with a real screenshot of the running application (or of mvn test).

8.5 Forex-Pos (1 point)

GET /forex/positions → OandaService.positions() → GET /v3/accounts/{id}/openTrades, path “trades”. forex.html, when page is positions and data is a JSON array, renders a table: Trade ID, Instrument, Units, Open price, Unrealized P/L — fulfilling “Prints the open positions in a table.”

[SCREENSHOT PLACEHOLDER] Figure 12 — Forex-Pos table of open trades.

Replace this box with a real screenshot of the running application (or of mvn test).

8.6 Forex-Close (2 points)

GET /forex/close shows a number field Trade ID (min 1). POST /forex/close calls OandaService.close(tradeId) which rejects non-positive ids and PUTs /v3/accounts/{id}/trades/{tradeId}/close with {"units":"ALL"} — close the given position at market.

[SCREENSHOT PLACEHOLDER] Figure 13 — Forex-Close: trade id entered and close confirmation from OANDA.

Replace this box with a real screenshot of the running application (or of mvn test).

OANDA endpoint summary

Menu	HTTP	Path
account	GET	/v3/accounts/{id}/summary
ActPrice	GET	/v3/accounts/{id}/pricing?instruments=
HistPrice	GET	/v3/instruments/{ins}/candles?count=10&price=M
Open	POST	/v3/accounts/{id}/orders
Pos	GET	/v3/accounts/{id}/openTrades
Close	PUT	/v3/accounts/{id}/trades/{tradeId}/close

9. Extra: Test menu, JUnit and MockMvc (+5 points)

Supplementary task: a form similar in complexity to the four-operation calculator from the Java Testing chapter; JUnit and MockMvc tests in comparable number; screenshots of test execution in the documentation. Forex Horizon implements this as the Test menu (not React).

Application form

GET /test and POST /test. Fields: two numbers (a, b), operation add/subtract/multiply/divide. CalculatorService uses BigDecimal. Division uses scale 8, HALF_UP, then stripTrailingZeros. Null operands, divide-by-zero, and unknown operations throw IllegalArgumentException; the controller catches Exception and shows model attribute error. Success puts result on the model. View: test.html.

[SCREENSHOT PLACEHOLDER] Figure 14 — Test menu: successful addition result.

Replace this box with a real screenshot of the running application (or of mvn test).

[SCREENSHOT PLACEHOLDER] Figure 15 — Test menu: divide-by-zero error message.

Replace this box with a real screenshot of the running application (or of mvn test).

JUnit — CalculatorServiceTest

Eight unit tests, comparable to a course calculator suite: addition, subtraction, multiplication, division (7/2 = 3.5), division by zero, unsupported operation (power), missing operand (null), negative numbers.

MockMvc — PageControllerTest

@WebMvcTest(PageController.class) with @MockitoBean for CalculatorService, OandaService, MnbService. Tests: GET / → view index; GET /test → view test; POST /test success expects model result 5; POST divide-by-zero expects model error; GET /soap → soap; GET /forex/close → forex. This proves MVC wiring without calling MNB or OANDA.

```
mvn test
# expected: CalculatorServiceTest and PageControllerTest both OK
# reports: target/surefire-reports/
```

[SCREENSHOT PLACEHOLDER] Figure 16 — IntelliJ or terminal screenshot of mvn test with all tests passed (mandatory for +5).

Replace this box with a real screenshot of the running application (or of mvn test).

10. GitHub version control and project method (2 + 3 points)

THE TASK (2 p): use github.com; do not upload only the finished application in one step; commit partial states in at least five steps; repository public; use your own name on Git. Project method (3 p): it must be visible which group member created each part. Help document:

GitHub-description-2026-07.docx (Desktop, branches, pull requests, fork, .gitignore).

Minimum commit story (do this on two accounts)

- Commit 1 — Maven Spring Boot skeleton and README.
- Commit 2 — Home page, fragments, Bootstrap theme, authors footer.
- Commit 3 — SOAP MNB client and Chart.js page.
- Commit 4 — OANDA service and Forex menus (can be split into more commits).
- Commit 5 — Test calculator, JUnit, MockMvc.
- Later — homework.jar at repo root, documentation PDF is submitted to Teams only (not required on GitHub).

Use two contributor accounts (or a fork + pull request as in the GitHub description's 2nd exercise) so the Contributors graph shows both students. Do not squash all work into a single "final" commit. Set visibility to Public (Settings → Change repository visibility). Put the URL in section 2.

[SCREENSHOT PLACEHOLDER] Figure 17 — Public GitHub repository page showing ≥5 commits and two contributors.

Replace this box with a real screenshot of the running application (or of mvn test).

[SCREENSHOT PLACEHOLDER] Figure 18 — homework.jar visible in the repository root on GitHub.

Replace this box with a real screenshot of the running application (or of mvn test).

11. Security: secrets, practice account, .gitignore

GitHub-description and THE TASK both forbid putting passwords and API tokens in Git. Forex Horizon keeps OANDA_API_TOKEN and OANDA_ACCOUNT_ID in the environment. .env is gitignored; .env.example contains only placeholders. application.properties references ${OANDA_API_TOKEN:} with an empty default.

Trading warning: OandaService can open and close market orders. Use only a practice account. Do not change app.oanda.base-url to the live API. Losses on a live account are possible if that URL is changed.

MnbService XML parser disables DTDs and external entities to reduce XXE risk when parsing SOAP.

12. Appendix — OANDA evaluator access and screenshot checklist

OANDA practice login (required in this PDF by THE TASK)

THE TASK: “In the documentation, give the URL address of the GitHub site, login details to the Oanda account.” Fill the fields for the teacher. Prefer a practice account created for evaluation. Do not commit this PDF with live tokens to a public GitHub repo; submit the PDF on Teams only.

Item	Fill before Teams upload
OANDA web login URL	https://www.oanda.com/ (practice)
Practice username / email	_____
Practice password	_____
Account ID	_____
API token (v20)	_____ (or how the teacher should obtain it)
API host	https://api-fxpractice.oanda.com

How each previous task is implemented (evaluation index)

THE TASK: describe in detail how and where you implemented the previous tasks; the work is evaluated based on this.

Requirement	How / where
Java 25	pom.xml java.version 25; compiler release 25
homework.jar + java -jar	spring-boot-maven-plugin finalName homework; copy to repo root

Responsive theme	Bootstrap 5.3.3 + site.css; described in §5
Company home	GET / → index.html fictional Forex Horizon
Names on every page	footer fragment + app.authors
SOAP + dates + chart	MnbService SOAP POST; soap.html table + Chart.js
Six Forex menus	PageController /forex/* + OandaService methods
Git ≥5 commits, public, two people	§2 and §10; evidence screenshots
Test extra	CalculatorService + two test classes; §9

Screenshot checklist (replace all placeholders)

- Java 25 version and jar startup
- Home desktop and mobile
- Navigation menu including Forex items
- SOAP form, table, Chart.js
- All six Forex pages with real practice-account data
- Calculator success and error
- mvn test green run
- Public GitHub URL, commit history, two contributors, homework.jar in root

What was not implemented

React menu extra (+5): not implemented. No H2 database, no Axios CRUD, no c:\data folder. The assignment allows this as optional. Forex Horizon stays a Thymeleaf MVC app as specified for the 25-point core plus the Test extra.

Submission reminder

- Export/save this file as Name-NeptunCode.pdf (example: GeorgeBrown-ABC123.pdf).
- Each student uploads only the PDF to Teams.
- Keep GitHub public with homework.jar and source.
- Replace every screenshot placeholder with a real capture of YOUR running app.
- Replace Student One/Two and GitHub/OANDA blanks.

End of documentation. After screenshots and personal data are filled, print/export and confirm the PDF is at least 10 pages as required by THE TASK.